Anexo de trazabilidad del uso de inteligencia artificial

PredictIT · Trabajo Final de Ingeniería · Patricio Pettini · B00072710-T1

Este anexo se genera con docs/trazabilidad/_generar-anexo.py a partir de AI-USAGE-LOG.md y PROMPTS-CRITICOS.md. Los dos archivos de texto son la fuente: se editan a mano durante el desarrollo y este documento se reconstruye entero cada vez, así no quedan las dos versiones diciendo cosas distintas.

Anexo A · Registro de uso de inteligencia artificial

Proyecto: PredictIT — Sistema de mantenimiento predictivo de equipos informáticos

Autor: Pettini, Patricio Ezequiel — Legajo B00072710-T1

Trabajo Final de Ingeniería — UAI, Facultad de Tecnología Informática

Propósito y alcance

Este anexo registra qué herramientas de inteligencia artificial se utilizaron durante el desarrollo del sistema, para qué tareas concretas, y qué se modificó sobre lo que esas herramientas propusieron.

El registro cubre la **fase de construcción del sistema**, desde el inicio del desarrollo. Las decisiones de arquitectura que surgieron de una consulta asistida se documentan además en `docs/adr/` y el detalle de las consultas en `docs/trazabilidad/PROMPTS-CRITICOS.md`.

Se registra **lo que efectivamente ocurrió**. Cuando una propuesta de la herramienta resultó incorrecta y hubo que corregirla, queda asentado: ése es justamente el dato que hace útil el anexo.

Herramientas utilizadas

Herramienta	Modelo / versión	Uso en el proyecto
Claude Code	Claude Opus 5	Análisis del documento y del proyecto de referencia, diseño del esquema de base, andamiaje de la solución, redacción de código y de documentación técnica.

> *Si en el transcurso del trabajo se incorporan GitHub Copilot, ChatGPT u otras herramientas, se agregan acá con el mismo criterio.*

Criterio de registro

Se asienta una entrada por **entrega o fase**, indicando:

1. **Tareas** para las que se usó la herramienta.
2. **Qué se aceptó** de lo propuesto.
3. **Qué se modificó o descartó**, y por qué.
4. **Cómo se validó técnicamente** el resultado.

No se registran las interacciones triviales (completado de línea, renombres, formato). Sí se registra toda intervención que haya afectado el diseño, el modelo de datos, la seguridad o el comportamiento del sistema.

E-00 · Análisis previo y definición del alcance técnico

Fecha: 02/09/2026 · **Herramienta:** Claude Code

Tareas

- Extracción y análisis del documento del TFI (157 páginas): requerimientos funcionales y no funcionales, casos de uso CU-001 a CU-013, casos de prueba CP-01 a CP-14, DER de negocio y de servicio, y las 16 pantallas propuestas.
- Análisis del proyecto de referencia de 3.º año (*Trabajo de Diploma*) para identificar el modelo de capas y los patrones que aplica la cátedra.
- Análisis de los manuales del proyecto de referencia para extraer la estructura exigida.

Qué se aceptó

El relevamiento de la estructura del proyecto de referencia: `IGenericDao<T>`, `IObjectMapper<T>`, `FactoryDao` / `FactoryBusiness`, `IUnitOfWork` con `SqlTransactRepository`, `SqlHelper` / `SqlHelperSeguridad`, y el esquema de permisos Usuario / Familia / Patente con contraseñas PBKDF2. Se adoptó como base del sistema.

Qué se modificó o descartó

- **El autor corrigió el documento fuente.** La herramienta había analizado una versión anterior y menos completa del TFI. El autor identificó el error y aportó la versión definitiva (v2.0, 10/07/2026), sobre la que se rehízo el análisis. Se verificó luego que el capítulo 10 coincidía en un 99,4 % entre ambas versiones y que la diferencia real era la sección 7.7.
- **El autor acotó el alcance de la documentación.** La herramienta propuso replicar la estructura documental completa del proyecto de referencia. El autor precisó que esa estructura no se exige, pero que **los manuales sí**, redirigiendo el análisis hacia esa carpeta.
- **Se descartó incorporar dígito verificador.** Se verificó contra el Manual de Programador de referencia que el proyecto anterior no lo implementaba y que no está entre los requerimientos de arquitectura del documento (Req. Arq. 001 a 005).

Validación

Comparación programática del capítulo 10 entre las dos versiones del documento (similitud 99,4 %), y lectura directa del código fuente del proyecto de referencia —no de su documentación— para confirmar los patrones efectivamente implementados.

E-01 · Decisiones de arquitectura

Fecha: 02/09/2026 · **Herramienta:** Claude Code

Detalle de las consultas: `docs/trazabilidad/PROMPTS-CRITICOS.md`

Tareas

Evaluación de alternativas para plataforma, motor y estrategia de acceso a datos, modelo multi-organización, integración con el servicio de IA y diseño del motor predictivo.

Qué se aceptó

El análisis comparativo de alternativas y la identificación de consecuencias negativas de cada opción, que se incorporaron a los ADR 0002 a 0008.

Qué decidió el autor

Las decisiones fueron del autor, en varios casos apartándose de lo que planteaba el documento o eligiendo la opción más costosa:

Decisión	Tomada por	ADR
C# / ASP.NET Core en lugar del Java / Spring Boot que fija el documento	Autor	0002
Dos bases físicas separadas, asumiendo la pérdida de integridad referencial	Autor	0003
ADO.NET con DAO explícito en lugar de EF Core, asumiendo el código repetitivo	Autor	0004
Multi-organización desde el modelo, ampliando el DER del documento	Autor	0005
Alcance de la IA: asignación **y** clasificación, descartando otras opciones propuestas	Autor	0006
Proveedor de IA conmutable, ante la falta de crédito de API	Autor	0006

Se descartaron explícitamente dos ampliaciones de alcance que la herramienta había puesto sobre la mesa —explicación del riesgo en lenguaje natural generada por IA, y asistente conversacional sobre el parque— por quedar fuera de lo que el documento sostiene.

Validación

Cada decisión se documentó en formato ADR (Nygard) obligando a explicitar alternativas descartadas y consecuencias negativas. Las decisiones que contradicen el documento se registraron además en `docs/CAMBIOS-DOCUMENTO.md` con la sección exacta a modificar.

E-02 · Modelo de datos y entorno de desarrollo

Fecha: 02/09/2026 · **Herramienta:** Claude Code

Tareas

Esquemas de `PredictIT_Negocio` y `PredictIT_Servicio`, datos iniciales, script de aplicación, contenedor de SQL Server, andamiaje de la solución .NET con sus seis proyectos, y pruebas de verificación del entorno.

Qué se aceptó

La estructura de tablas derivada del DER del documento, los índices orientados a las consultas del motor predictivo (`Incidencia` por equipo y fecha) y el trigger `INSTEAD OF UPDATE, DELETE` que hace inmutable la bitácora en la base y no en la aplicación.

Qué se modificó o descartó

Tres defectos del esquema propuesto se detectaron recién al ejecutarlo contra el motor real, no en la revisión:

1. `Organizacion.plan` y la tabla `Backup` usaban **palabras reservadas de T-SQL**. Renombrados a `plan_comercial` y `Respaldo`.
2. Los **índices filtrados** (`WHERE ... IS NOT NULL`) fallaban porque `sqlcmd` deja `QUOTED_IDENTIFIER` en `OFF`. Corregido en el script de aplicación.
3. `TipoEvento.criticidad` estaba declarado `VARCHAR(10)` y el valor `'ADVERTENCIA'` tiene 11 caracteres. Truncamiento silencioso evitado por el error de inserción.

Es el punto que mejor ilustra por qué la validación por ejecución es obligatoria: los tres defectos eran invisibles a la lectura del script.

- **Intervención manual del autor en el entorno.** Docker Desktop fallaba al iniciar por sockets residuales de una sesión anterior. La corrección la ejecutó el autor directamente sobre su equipo.

Validación

- Los cuatro scripts se ejecutan contra SQL Server 2022 real, no se revisan a ojo. Son idempotentes y se corrieron varias veces.
- **26 pruebas automatizadas** verifican: existencia de las 20 tablas, carga de datos iniciales, que ninguna contraseña esté en texto plano, que el trigger de inmutabilidad de la bitácora efectivamente rechace un `UPDATE`, y que el historial de demostración alcance el umbral que dispara la regla de recurrencia.
- Consulta de verificación sobre la base cargada confirmando la distribución buscada de incidencias por equipo.

E-03 · Diseño de interfaz

Fecha: 02/09/2026 · **Herramienta:** Claude Code

Tareas

Definición del sistema de diseño —tipografía, paleta con criterio semántico, escala de espaciado— y construcción de pantallas.

Qué se modificó o descartó

El autor descartó el enfoque propuesto. La herramienta había empezado a construir las 16 pantallas directamente en la sesión. El autor evaluó el resultado como insuficiente y redirigió el trabajo: en lugar de seguir produciendo pantallas, se elaboró una especificación de diseño completa para trabajar el diseño en una herramienta específica.

Se conservaron las tres pantallas ya construidas únicamente como referencia del vocabulario visual, no como entregable.

Validación

La validación del diseño es la revisión del autor sobre el resultado, y la verificación de que las pantallas construidas se correspondan con los datos reales del sistema. Ambas se completaron en **E-05**: el diseño se aprobó y las pantallas implementadas se recorrieron contra los datos del sistema, no contra datos de ejemplo. La verificación encontró un desvío del propio diseño —el mensaje del plan afirmaba una conveniencia económica que la base contradice— y quedó asentado ahí.

E-04 · Implementación del backend (corte vertical)

Fecha: 02/09/2026 · **Herramienta:** Claude Code

Tareas

Construcción del corte vertical completo —autenticación, consulta de activos y situación del plan— atravesando las cinco capas: `Domain`, `DAO`, `BLL`, `Service` y `Api`. Incluye el Composite de permisos, el aislamiento por organización en el DAO, el hash PBKDF2 y la suite de pruebas.

Qué se aceptó

- La estructura de capas y los patrones del proyecto de referencia (`IGenericDao<T>`, Abstract Factory, Unit of Work, Composite), que ya estaban decididos en los ADR.
- El esqueleto de los DAO y de los mapeadores, revisado consulta por consulta.

Qué se modificó o descartó

El registro más útil de este incremento son los defectos que la revisión del código **no** detectó y que sólo aparecieron al ejecutar. Es la evidencia concreta del riesgo que introduce la velocidad del código asistido:

Defecto	Cómo se detectó	Por qué la lectura no alcanzaba
PLAN y BACKUP son palabras reservadas de T-SQL	Al ejecutar el script	El SQL se lee correcto; falla el parser
`TipoEvento.criticidad VARCHAR(10)` truncaba `'ADVERTENCIA'` (11 caracteres)	Al ejecutar el seed	Requiere contar caracteres contra el largo declarado
Índices filtrados rechazados por `QUOTED_IDENTIFIER`	Al ejecutar el script	Es una opción de sesión de `sqlcmd`, no del script
Dependencia circular `IContextoSesion` a `ISeguridadService` a `IFactoryDao` a `IContextoSesion`	Al hacer el primer pedido **autenticado**	El login funcionaba: el camino anónimo cortocircuita el ciclo
Paquetes NuGet de una versión incompatible se agregaban **en silencio**	Al compilar y al arrancar	`dotnet add package` no reportó error

La dependencia circular es el caso más relevante: el sistema compilaba, el login andaba, y el defecto sólo se manifestaba en el segundo pedido. Se resolvió separando `IFactoryDaoSeguridad`, que no recibe contexto porque sus tablas no tienen `id_organizacion`.

También se reescribió código propuesto por la herramienta que funcionaba pero era ilegible: la resolución de la organización activa (un ternario anidado) y una caché de planes escrita con una construcción sin sentido (`is var _ ?`).

Validación

- **62 pruebas xUnit en verde**, no sólo compilación: hash PBKDF2 contra los cuatro valores literales del seed, resolución de permisos anidados con familia dentro de familia, diamante y ciclo, y una prueba por operación para el aislamiento entre organizaciones.
- Los scripts de base de datos se **ejecutaron** contra SQL Server en contenedor. No se dieron por válidos por revisión.
- La API se probó con pedidos HTTP reales, verificando los 401 y 403 esperados.

E-05 · Implementación del frontend

Fecha: 02/09/2026 · **Herramienta:** Claude Code

Tareas

Implementación en React 19 + TypeScript del diseño producido en E-03: cliente HTTP, manejo de sesión y patentes, layout, listado de activos con filtros, detalle de equipo y pantalla de organización con la situación del plan.

Qué se aceptó

La estructura de componentes y el cliente HTTP, con dos decisiones tomadas por el autor y no por la herramienta: el token va en `sessionStorage` y no en `localStorage` —en una PyME sin área de IT las máquinas se comparten— y la interfaz esconde lo que el usuario no puede hacer, pero la autorización la decide la API.

Qué se modificó o descartó

Se descartó una afirmación incorrecta de la herramienta sobre el propio modelo de negocio. Al construir la pantalla del plan, la herramienta afirmó que con 52 equipos convenía el Plan Estándar. Verificado contra la base: el Plan Inicial cuesta $101.500 y el Estándar $102.900; el Estándar recién conviene a partir de 57 equipos. La corrección mejoró el sistema: la pantalla muestra la cuenta real, incluso cuando el resultado es que **no** conviene cambiar al plan más caro. Un sistema que empuja al plan caro cuando la cuenta dice lo contrario es un problema de confianza, no un error de cálculo.

Defectos propios encontrados en la verificación:

Defecto	Cómo se detectó
«1 equipos registrados» en lugar del singular	Recorrido manual en el navegador
El montaje disparaba **dos** consultas idénticas (el filtro inicial y el debounce asentándose con el buscador vacío)	Al escribir la prueba del debounce
La API arrancaba con una clave de firma más corta que los 32 bytes que exige HS256, y fallaba recién al emitir el primer token	Al documentar la plantilla de configuración

El tercero es el más significativo: no era un defecto visible, era una validación faltante. Se agregó el chequeo al arrancar, de modo que una clave corta produce un error que dice cuál es el problema en lugar de una excepción de la librería.

Se descartaron además dos pruebas que la herramienta había escrito y que **no probaban nada**: una usaba `vi.doMock` después de importar el módulo (por lo que la simulación nunca se aplicaba) y la otra afirmaba `>= 0`, que es verdadero siempre.

Validación

- **24 pruebas Vitest en verde**, incluidas las tres que cubren los defectos de arriba y las dos ramas del mensaje del plan (conviene / no conviene).
- Recorrido manual de todos los flujos en el navegador, comparando cada pantalla contra el diseño de E-03.
- `npm run build` limpio y **0 vulnerabilidades** reportadas por `npm audit`.
- Una prueba del frontend falló por un defecto de la **propia simulación** —devolvía una función nueva en cada render, lo que disparaba una consulta por render—. Queda asentado porque ilustra el riesgo inverso: una prueba mal escrita puede acusar al código correcto.

E-06 · Motor predictivo e integración de IA

Fecha: 02/09/2026 · **Herramienta:** Claude Code

Tareas

Motor de análisis predictivo con las cinco reglas del documento (RF-11, RF-12), proveedor de IA conmutable con disyuntor (ADR 0006 y 0009), y el flujo completo de incidencias: alta, triage del texto libre, asignación automática y asignación de respaldo. Backend y pantallas.

Qué se aceptó

La estructura general del motor —una estrategia por tipo de regla, score ponderado— que ya estaba decidida en el ADR 0007.

Qué decidió el autor

Cuatro decisiones que la herramienta no propuso y que cambian el comportamiento del sistema:

1. **La fecha de referencia entra por parámetro; el motor no lee el reloj.** Sin esto no existe caso de prueba posible: el resultado cambiaría según el día en que se corran las pruebas. Es la decisión que hace verificable todo el capítulo.
2. **Una regla que no se puede evaluar por falta de datos se excluye del promedio ponderado, en lugar de aportar cero.** Si aportara cero, el equipo con la ficha incompleta aparecería *menos* riesgoso que el que está bien cargado, que es exactamente al revés de lo que corresponde.
3. **La criticidad del equipo multiplica el score, no lo suma.** Sumando, un servidor nuevo y sano aparecería siempre en riesgo. Multiplicando, cero sigue siendo cero.

4. **Un disyuntor por organización y no uno global.** Si el proveedor falla para un cliente —su clave venció, su cuenta se quedó sin crédito— el circuito no puede abrir para todos los demás. El ADR 0009 no distinguía este caso.

Qué se modificó o descartó

Se corrigió una falta de honestidad en la trazabilidad que la herramienta introdujo. El proveedor determinístico resuelve por coincidencia de términos y puntaje, no con un modelo de lenguaje. La primera implementación registraba esas decisiones con origen `IA`, porque era el único valor que la base admitía. Eso falsea justamente lo que el sistema tiene que poder demostrar en la defensa: qué decidió una IA y qué decidió una regla. Se agregó `HEURISTICA` como origen —en `RecomendacionAsignacion` y en `Incidencia.tipo_asignacion`— con migración con guarda, y la pantalla distingue «Decidida por el proveedor de IA» de «Decidida por reglas automáticas».

Se rechazó una confianza inflada. La primera versión del proveedor simulado devolvía confianzas altas. Se bajaron a 0,20 y 0,55: es coincidencia de palabras, y decir lo contrario sería mentirle al técnico que después decide si acepta la sugerencia.

Defectos que la revisión del código **no** detectó y que sólo aparecieron al ejecutar o al recorrer las pantallas:

Defecto	Cómo se detectó	Por qué la lectura no alcanzaba
El join de técnicos pasaba por `Usuario_Organizacion`, que sólo tiene filas del Partner: devolvía cero técnicos y ninguna incidencia se asignaba	Llamada HTTP real al catálogo	La consulta se lee correcta; el problema es que la pertenencia a una organización se expresa de dos formas distintas en el modelo
`UsuarioDao.PorOrganizacion` tenía el mismo sesgo: el Partner quedaba afuera y su nombre salía vacío	Nombre en blanco en la respuesta del CP-14	Requiere cruzar dos consultas escritas en momentos distintos
`RequierePatente` exigía una sola patente; `INCIDENCIA_VER_TODAS` y `VER_PROPIAS` son distintas y ninguna implica a la otra	403 al administrador	El código se lee razonable: el error está en el modelo de permisos, no en la sintaxis
Asignar un técnico no movía el estado: todo quedaba en «Pendiente de asignación»	Recorrido en el navegador	El dato estaba bien guardado; lo que faltaba era una transición del ciclo de vida

Validación

- **131 pruebas en verde** (69 nuevas): 22 del motor con escenarios de fecha fija, 28 del disyuntor y los proveedores, y 19 de integración contra SQL Server.

- **CP-14 demostrado sobre el sistema corriendo**, no sólo en pruebas unitarias: con el proveedor forzado a fallar, el circuito abre al quinto fallo, deja de llamar al servicio, y las incidencias quedan asignadas por respaldo y marcadas para revisión.
- El motor evaluó los **47 equipos reales de la base en 913 ms**, generando 59 alertas, con el score de cada equipo desglosado regla por regla. La segunda corrida no duplicó ninguna alerta.
- Se cargó una incidencia desde el formulario y el sistema la clasificó «Periféricos · prioridad Alta» y la asignó al técnico con especialidad «Impresión» nivel 3 que además tenía dos casos previos en ese mismo equipo.

Una limitación que conviene decir y no disimular: la demostración del CP-14 con el proveedor simulado no muestra el ahorro de latencia que justifica el disyuntor, porque el fallo simulado es instantáneo. Lo que la demostración sí prueba es la transición de estados y que se deja de llamar al proveedor. El ahorro de tiempo sólo se puede medir contra un servicio real que efectivamente tarde.

E-07 · Auditoría de arquitectura y cierre de requisitos

Fecha: 02/09/2026 · **Herramienta:** Claude Code

Tareas

Verificación de los cinco Req. Arq. y los siete CU.Arq. del modelo de referencia de 3.º año contra el sistema construido, y corrección de lo que faltaba: multiidioma completo, cifrado reversible, `Patente.TipoAcceso`, filtro de bitácora por usuario y pantalla de errores.

Qué se aceptó

La lectura del material de referencia y el armado de la tabla de estado por requisito, que sirvió para no discutir de memoria.

Qué decidió el autor

Que la auditoría se hiciera contra datos, no contra el código. Es la decisión que dio resultado. Los dos hallazgos serios no eran errores sino **ausencias**, y una ausencia no se ve leyendo:

1. **La bitácora no registraba ningún evento de negocio.** Tenía 37 inicios de sesión y ninguna alta de incidencia. Ocho de los nueve códigos que el sistema asentaba no existían en el catálogo, y el servicio los descarta en silencio por diseño —perder una línea de auditoría es malo, abortar el alta de un equipo por no poder escribirla es peor—. Cada llamada estaba correctamente escrita. Apareció al contar filas por tipo de evento.

2. **El motor predictivo no se disparaba solo.** Lo reportó el autor mientras la herramienta trabajaba en otra cosa, y la evidencia disponible parecía contradecirlo: el motor pasaba sus pruebas y la API devolvía scores correctos. Buscar quién lo invocaba mostró que nadie, salvo el botón de la pantalla.

Que un código nuevo mal escrito no pueda repetir el primer defecto. No alcanzaba con completar el catálogo: se pasaron las llamadas a constantes —una constante mal escrita no compila— y se agregaron dos pruebas, una que verifica por reflexión que cada constante exista en la base y otra que rechaza literales sueltos en las llamadas a bitácora.

Que el barrido predictivo no corra como superusuario. Un proceso automático con todos los permisos es un proceso que puede hacer cualquier cosa el día que tenga un error. Lleva un contexto de sistema con la única patente que necesita, y sin usuario: la bitácora tiene que decir que lo hizo el sistema y no atribuírselo a una persona que no estaba.

Que la clave de API se guarde cifrada por organización y no se pueda leer. La propuesta inicial era más simple —dejarla en la variable de entorno, que ya funcionaba—. El problema no era técnico sino de producto: con la clave en el entorno, todas las organizaciones del plan Partner comparten la credencial de quien opera el servidor. Y no hay endpoint que la devuelva: una credencial que puede salir del servidor es una credencial que se puede filtrar por cualquiera de los caminos por los que sale.

Que la pantalla de errores salga de la bitácora y no de una tabla nueva. Un segundo registro de lo mismo se desincroniza del primero.

Verificación

- 171 pruebas del backend y 24 del frontend, en verde.
- La clave de API queda en la base como `AESGCM$v1$…`, sin rastro en claro ni en la configuración ni en la bitácora.
- La bitácora de un técnico pasa de 56 filas a 7 al filtrar por su usuario, y todas son de esa persona.
- Un técnico recibe 403 en la pantalla de errores, en la configuración de IA y al intentar cargar una clave.
- El barrido programado corrió sobre las dos organizaciones —49 equipos— y dejó su rastro en bitácora; una incidencia registrada disparó la reevaluación de su equipo.

E-08 · Cierre de los casos de uso faltantes y de la evaluación financiera

Fecha: 03/09/2026 · **Herramienta:** Claude Code

Tareas

Ciclo de vida completo de la incidencia, cambio de estado del equipo, reportes exportables a PDF, historial del parque, cierre de sesión con rastro y restauración de respaldos. El frontend de todo eso. Valor terminal en el presupuesto financiero.

Qué se aceptó

La técnica de auditoría —buscar lo declarado y no usado— y la implementación mecánica una vez decidido el diseño.

Qué decidió el autor

El formato de los reportes. La propuesta era CSV, por ser lo más barato y sin dependencias. El autor eligió PDF, y tenía razón para lo que se va a mostrar: un CSV no lleva encabezado ni los filtros impresos, y no se ve como un entregable.

El horizonte de la evaluación financiera. Primero pidió bajarlo a tres años, con un argumento sólido —en tecnología y con la IA incorporándose, la proyección del cuarto año no es creíble—, y después decidió mantener los cuatro con valor terminal. Las dos decisiones son suyas.

Que el múltiplo del valor terminal fuera el piso del rango y no el que cierra el número. A 1,5× el VAN queda en −48 millones y se vuelve positivo a 2,0×, que está dentro del rango de mercado. La tentación era mover el múltiplo hasta que diera positivo; el criterio fue tomar el piso, declarar el resultado negativo, y poner la sensibilidad al lado para que la conclusión no dependa de haber elegido bien un número que nadie puede conocer.

Que la lectura del resultado se genere con fórmulas. Un texto fijo que afirme «el VAN es positivo» sobrevive al cambio de hipótesis que lo vuelve falso, y es el primer error que un jurado encuentra.

Defectos propios de esta etapa

Cuatro, y los cuatro se encontraron ejecutando:

1. **El interbloqueo del botón «Resuelta»** (PC-18). Diseño propio, dos lados correctos por separado, imposible de ver sin usar la pantalla.
2. **`FF_TASA` con un desplazamiento contado a mano** sobre el orden de la lista de indicadores del flujo de fondos. Agregar dos indicadores lo dejó apuntando a otra fila y los VAN salieron `#VALUE!`. Ahora la tasa va primera en la lista y agregar indicadores no mueve nada.
3. **La tabla de sensibilidad calculaba el valor terminal como delta** contra el múltiplo vigente, así que en la columna del múltiplo actual la diferencia era cero y mostraba el VAN *sin* valor

terminal — es decir, el número equivocado justo en la columna que un lector compara contra el indicador de arriba.

4. **`TEXT` con separadores decimales** se rompe según la configuración regional: la conclusión generada decía `-$ 47698378,610` y `05 %`.

Verificación

- 238 pruebas de backend y 31 de frontend, en verde. La cifra la controla la integración continua en cada push, así que no se escribe de memoria.
- El ciclo completo de una incidencia recorrido dos veces: por la API, con sus cuatro rechazos —salto inválido, resolver sin diagnóstico, resolver sin solución, tocar una cerrada—, y después por la pantalla, resolviendo la incidencia #24 como el técnico asignado.
- De los tres técnicos, sólo el asignado pudo atender la #24.
- Los tres PDF generados y abiertos: el del parque, tres páginas con encabezado, filtros impresos, resumen y cabecera repetida.
- De los cuatro perfiles, el solicitante recibe 403 en los reportes y los otros tres pueden, que es lo que dice el seed.
- El libro de Excel recalculado con LibreOffice: sin un solo error de fórmula, y los indicadores leídos de las celdas y no estimados.

E-09 · Ampliación del alcance de la IA: guía de reparación

Fecha: 09/09/2026 · **Herramienta:** Claude Code

Tareas

Tercera función del proveedor de IA, junto al triage y la asignación: una guía de diagnóstico y reparación paso a paso para una incidencia (RF-15, CU-018). Contrato, las dos implementaciones, el disyuntor, el método de negocio con su control de acceso, el endpoint, los tipos de evento de bitácora y las pruebas.

Qué se aceptó

La estructura del corte: la misma que ya tenían las otras dos operaciones —contrato en `IProveedorIA`, implementación real y simulada, disyuntor por decorador—, que es lo que hace que la función nueva no invente un camino propio.

Qué decidió el autor

1. **Antes de pedir la función, pedir la verificación de si ya existía.** La consulta no fue «implementá la guía» sino «confirmá si esto está y después decido». No estaba: el

proveedor tenía `ClasificarAsync` y `RecomendarTecnicoAsync`, y los campos de diagnóstico y solución los escribía el técnico a mano. Recién con eso confirmado se decidió sumarla.

2. **El sistema no toca el equipo del cliente.** Es el límite del alcance y se sostuvo: la guía la ejecuta una persona. Está escrito en el prompt del proveedor real para que no proponga acciones remotas. Sumar ejecución sería otro producto, con otra responsabilidad.
3. **La guía no se persiste.** Depende del historial del equipo en el momento en que se pide. Guardarla invitaría a leer meses después una guía armada con otro historial y creerle. Queda el asiento en bitácora, que además es lo que alimenta el costo por consulta del presupuesto.
4. **Se pide, no se genera sola.** Consultar al proveedor en cada alta sería pagar por todas las incidencias para que sirva en algunas.

Qué se corrigió de lo producido

La primera versión de las recetas del proveedor simulado tenía, en la categoría Periféricos, «reinstalar el controlador» después de «vaciar la cola de impresión» y sin declarar riesgo. La regla que el propio diseño se había puesto —los pasos con riesgo van al final— quedaba violada. Se declaró el riesgo real de ese paso —se pierden las preferencias guardadas del dispositivo— en lugar de aflojar la regla. Lo encontró la prueba que recorre las seis categorías, que se escribió justamente para eso.

E-10 · Dos módulos de gestión: el plan de mantenimiento y la facturación

Fecha: 10 y 11/09/2026 · **Herramienta:** Claude Code

Tareas

Dos módulos completos, de la base a la pantalla: el plan de mantenimiento preventivo con su agenda (RF-16, CU-019 y CU-020) y la facturación del servicio (RF-17, CU-021). Cuatro tablas, dos DAO, dos business, dos controladores, tres pantallas, cincuenta y cinco pruebas, los datos de demostración y el volcado al documento.

Qué se aceptó

El reparto en capas y los patrones ya establecidos: los dos módulos entran por `FactoryDao` y `FactoryBusiness`, el filtro por organización vive en el DAO, y las pantallas reusan los

componentes y la capa de tokens que ya existían. No se inventó ninguna estructura nueva, que es lo que hace que el sistema siga leyéndose parejo.

Qué decidió el autor

1. **La pregunta fue si el sistema era demasiado simple, no «agregá módulos».** La duda era propia: el trabajo podía parecer pobre en gestión al lado de un sistema de ventas. Lo que se pidió fue contrastar ese juicio antes de sumar nada, porque agregar pantallas por volumen es la forma más rápida de que un trabajo parezca más grande y sea peor.
2. **Los dos módulos se eligieron por los huecos que cerraban, no por tamaño.** El motor predictivo recomendaba «programar el mantenimiento preventivo» y el sistema no tenía con qué programarlo: la recomendación terminaba en un papel. Y el plan de negocio define un modelo de ingresos por abono más equipos administrados que el sistema calculaba y no registraba. Los dos huecos existían antes de la pregunta.
3. **Con plan cargado, «vencido» lo define el plan y no el umbral de la regla.** Es el cambio de fondo del primer módulo y se decidió explícitamente: un equipo con el preventivo agendado para la semana que viene no es un equipo descuidado, aunque hayan pasado 250 días del anterior. El umbral queda para los equipos sin plan y para pesar el atraso.
4. **El comprobante no pretende ser fiscal.** No hay CAE ni integración con AFIP, y por eso la entidad se llama `Comprobante` y no `Factura`. Declararlo es preferible a un módulo que aparenta lo que no hace.
5. **Las pruebas se escribieron antes del cambio de la regla, y se comprobó que fallaran.** Seis pruebas, dos en rojo sobre el código viejo. Una prueba que nunca se vio fallar no prueba nada.

Qué se corrigió de lo producido

Un defecto de diseño de base propio, encontrado usando el sistema. La unicidad del correlativo del comprobante estaba escrita como `UNIQUE (id_organizacion, numero)`, con `numero` nulo mientras el comprobante es borrador. SQL Server trata el `NULL` como un valor más en una restricción de unicidad, así que la organización admitía **un solo borrador** y el segundo fallaba con un error del motor. Apareció al facturar dos meses seguidos a mano, no en las pruebas: los dobles no reproducen una restricción que vive en la base. Se corrigió con un índice único filtrado y quedó una prueba de integración que lo cubre.

Una prueba intermitente que se persiguió en lugar de darla por ruido. El CP-15 falló una vez y no volvió a fallar en quince corridas. El asiento de bitácora se hace dentro de un `catch` que se traga cualquier excepción —a propósito: no se aborta el alta de un equipo porque no se pudo escribir el log—, y sin reintento eso convierte un fallo pasajero de la base en un agujero de

auditoría silencioso, justo en los momentos de más concurrencia, que son los que más se auditan. Se agregó el reintento con sus tres pruebas.

Dos defectos de interfaz encontrados mirando las capturas. El menú lateral marcaba «Mantenimientos» y «Agenda» a la vez, porque una ruta es prefijo de la otra; poner `end` en todos los ítems rompía el caso contrario —en la ficha de un equipo el menú dejaba de señalar dónde estaba uno—, así que se resolvió con la coincidencia más específica y cuatro pruebas. Y el alcance de un plan por tipo decía «Todos los Servidor»: los nombres de los tipos salen de un catálogo que carga el usuario, así que no se puede concordar con ellos.

Verificación

Las reglas del ciclo del comprobante contra dobles; el SQL contra la base real en cuatro pruebas de integración; y los dos módulos probados a mano sobre el sistema corriendo, que es donde aparecieron dos de los tres defectos de arriba. Las dos afirmaciones que importaban del motor predictivo se comprobaron sobre datos reales y no sólo con dobles: un equipo con el trabajo vencido pasa a decir la fecha agendada y los días de atraso, y otro con 250 días desde el último preventivo pero con el próximo agendado deja de figurar como vencido.

E-11 · El frontend se rehace contra el prototipo

Fecha: 15/09/2026 · **Herramientas:** Claude Code y la skill

`ui-ux-pro-max` (nextlevelbuilder, MIT), instalada fuera del repositorio

Tareas

Llevar la interfaz a `docs/design/prototipo-completo.html`, en incrementos con su verificación: la capa de tokens, el armazón de la aplicación, los contadores de la navegación —que necesitaron endpoint propio— y las tablas y formularios.

Sobre la segunda herramienta, que es lo nuevo de esta entrada

`ui-ux-pro-max` es una base de datos consultable de criterios de diseño: 79 estilos, 192 paletas, 74 pares tipográficos y 119 reglas de UX con su fuente —Material Design, Apple HIG, WCAG— y su severidad. No genera el código del sistema: produce HTML con Tailwind, que este proyecto no usa.

Se usó como lista de control y no como generador. Se instaló en el perfil del usuario y no en el repositorio, así que no forma parte del entregable; queda asentada acá porque influyó en el resultado, que es lo que esta bitácora tiene que poder decir.

Antes de instalarla se leyeron sus scripts, porque los ejecuta la herramienta asistente: usan sólo biblioteca estándar de Python, sin red ni subprocesos. De las siete skills que trae el repositorio se copió una sola; las otras seis —logotipos, banners, identidad corporativa— no tienen que ver con el sistema y una de ellas llama APIs de imágenes de terceros.

Qué se aceptó

De sus 48 reglas de severidad alta, la mayoría ya estaba cubierta y eso también es un resultado: esqueletos de carga con `role="status"`, errores con `role="alert"`, estados vacíos con su acción, contraste AA verificado por script, `prefers-reduced-motion` respetado, y el contenido generado por IA rotulado como tal.

Una no lo estaba, y es la que justificó la herramienta: **el campo rechazado por el servidor no estaba enlazado con el mensaje que lo explica**. En pantalla se veía el borde rojo y el texto debajo, así que parecía resuelto; lo que faltaba —`aria-invalid` y `aria-describedby`— no se ve. Quien usa un lector de pantalla llegaba al campo, lo oía como cualquier otro, y el motivo quedaba suelto sin dueño.

Qué decidió el autor

Dónde vive la herramienta. Se instaló en el perfil y no en el repositorio: el entregable no se llena de archivos de terceros, y la trazabilidad se resuelve con esta entrada y no con 3,7 MB de CSV versionados.

Qué del prototipo no se implementa. La maqueta trae una barra superior para cambiar de rol, de tema y de densidad. Existe para poder mostrar las variantes en un HTML suelto; en el sistema el rol lo da la sesión y el tema y la densidad viven en Apariencia, así que esa barra no se construye.

El prototipo no manda sobre el RNF-07. Tres de sus pares de color no llegaban al piso de contraste de la WCAG AA. Se corrigieron moviendo sólo la luminosidad y dejando el tono, que es lo que el prototipo quería decir.

Qué se corrigió de lo producido

- El `--fuente-mono` de la capa de tokens apuntaba a Cascadia Mono mientras el `index.html` descargaba IBM Plex Mono. La fuente se bajaba en cada carga y el navegador dibujaba con otra.
- `_a-pdf.py` buscaba los `.docx` de las entregas parciales en dos carpetas que dejaron de existir al ordenar las entregas dentro de `ENTREGABLES`. Devolvía cero archivos sin avisar, y cuatro PDF se venían entregando viejos. Lo levantó el control de alineación, no la herramienta.

- La primera versión de la prueba del contador de navegación comparaba el número del solicitante contra el del administrador. Pasaba sola y fallaba en la suite completa: dependía de cuántas incidencias tuviera el seed en ese momento. Se reescribió comparando el contador contra el listado que **esa misma persona** ve, que es lo que la prueba quería afirmar.

Verificación

Cada incremento con su contraste verificado por `_contraste.py`, sus pruebas en verde y los 18 controles de `_verificar-alineacion.py`. Las guardas nuevas se comprobaron en rojo antes de darlas por buenas: se quitó el recorte por usuario del contador y se quitó el enlace entre campo y mensaje, y en los dos casos la prueba correspondiente falló.

E-12 · La segunda entrega de negocio: la devolución y la metodología de cierre

---

Fecha: 08 y 14/10/2026 · **Herramienta:** Claude Code

Tareas

Aplicar las cinco correcciones obligatorias de la devolución de la Entrega 1 —aprobada de manera condicional—, adoptar las definiciones metodológicas que la cátedra fijó después para el cierre de la evaluación financiera, incorporar la plantilla de flujo de fondos que publicó el 14/10 y armar la carpeta de la segunda entrega de negocio. Ocho deltas, D-82 a D-89, todos asentados en `CAMBIOS-DOCUMENTO.md`.

La entrega se arma con un guion propio, `docs/negocio/_entrega-2.py`, que **reusa el de la primera** en lugar de copiarlo: todo el armado del Word —la plantilla extraída del documento del Trabajo Final, el volcado del Markdown, la numeración de títulos, la portada— y el recorte del libro son los mismos, y lo único que cambia es qué documento, qué nombre de archivo y qué hojas. Dos copias del mismo formato es tener una que envejece sin que nadie la mire.

Qué se aceptó

- **Las empresas enteras.** La proyección comercial mostraba decimales como expectativa matemática. Pasa a cantidades enteras con arrastre: el flujo del mes es la diferencia entre dos acumulados redondeados, de modo que los errores no se suman. El efecto sobre el valor actual neto fue del −0,03 %, que es lo que validó el método.
- **El tráfico sale de sus causas.** Tres fuentes con comportamientos distintos: pauta que se apaga al dejar de pagarla y sigue al dólar, contenido que construye un stock con techo, y

referidos que entran como prueba directa porque quien llega recomendado no navega el sitio.

- **La capacidad operativa, medida.** Una hoja nueva convierte la cartera en horas y las horas en personas.
- **Los parámetros de la cátedra** —inflación 28/20/15 y tasa de corte del 40,0 %— se adoptaron tal cual, aunque el proyecto habría elegido una prima de riesgo de 20 puntos y no de 12. La diferencia se declara en la hoja de Hipótesis en lugar de disimularse.

Qué decidió el autor

Cuándo medir los umbrales. La consigna fija los tres targets «al cierre del Año 3» y, unas líneas antes, autoriza a ajustar el marco temporal cuando el negocio es más largo, evaluando «al finalizar el período evaluado». Nuestro horizonte es de cuatro años y los dos cortes no dan lo mismo: los tres umbrales se alcanzan en ambos, pero el retorno acumulado pasa del 192 % al año 3 al 653 % al 4. **Se declaran los dos** en lugar de elegir el que mejor queda, y se explica por qué difieren: el año 4 es el ejercicio en que el resultado operativo se da vuelta, así que esa distancia es exactamente lo que aporta el último ejercicio.

Qué se corrige después de medir y qué no. La hoja de capacidad mostró que la plantilla de soporte no alcanzaba desde el mes 28 y el calendario contrataba en el 42. Se adelantó el alta, y eso costó 17,9 millones de valor actual neto y 34,6 de capital. La alternativa era mover los parámetros de carga hasta que la plantilla alcanzara; está dicho en `CAMBIOS-DOCUMENTO.md` que no se hizo.

Que el escenario pesimista vuelva a dar negativo no se maquilla. Durante la preparación llegó a +4,4 millones y las dos correcciones posteriores lo devolvieron a −14,2. El apartado 8.10 se reescribió para decirlo, con las tres correcciones que lo movieron nombradas una por una.

Qué se corrigió de lo producido

- **Una afirmación del documento era falsa y la devolución la marcó.** Decía que cada incorporación de personal entra cuando la facturación puede pagarla. El modelo dice lo contrario: la segunda persona de soporte entra en el mes 28 y el resultado de ese mes es de −399.620 pesos. Las tres frases que lo afirmaban —en 7.6, 8.8 y 9.3— pasan a declarar las dos condiciones reales.
- **El déficit de soporte estaba mal contado.** Se había escrito «once meses» sumando seis del primer año, cuando todavía no hay posición dedicada y la carga la cubren los socios, con cinco del cierre del horizonte. Son dos situaciones distintas; el número correcto es cinco y la corrección quedó asentada en el delta en lugar de reemplazar el texto en silencio.
- **El libro quedó irreproducible por una restricción no documentada de Excel.** Una constante de texto dentro de una fórmula no puede pasar los 255 caracteres, y los párrafos de lectura

de este modelo son exactamente eso. Excel rechaza el archivo entero sin decir en qué celda. El generador ahora parte los literales largos y, antes de guardar, recorre todas las hojas y falla nombrando la celda.

- **Un indicador apuntaba a la columna equivocada.** El párrafo que explica el desvío de la tasa interna de retorno leía la columna de estado en lugar de la del valor y devolvía `#VALUE!`.
- **Las métricas de suscripción de 6.5.3 habían quedado viejas** al cambiar el presupuesto de marketing y la lista de precios: el valor del cliente, el costo de adquirirlo y su relación. Los márgenes por plan se verificaron contra la hoja «Planes» y no cambiaban.
- **El alcance de la entrega estaba corto.** La carpeta se armó con diecinueve de las veintitrés hojas, dejando los tres escenarios y los planes de contingencia para la final. Dos declaraciones del propio proyecto no coincidían: el README de negocio pone los escenarios en la Entrega 2 y la tabla de calendario del LEEME de la primera entrega los pone en la final. Se resolvió por el README, que es el más específico, y por un argumento que no depende de ninguno de los dos: el documento que acompaña la entrega ya trae los apartados 8.10 «Escenarios de Riesgo» y 8.11 «Plan de Contingencia», y entregar el documento que los describe sin las hojas que los calculan es incoherente. Lo levantó una pregunta del autor —«¿está todo?»—, no un control, y por eso ahora hay un control: `_verificar-entrega-2.py`.
- **El ROI estaba mal calculado, y lo destapó la plantilla de la cátedra**, que trae la fórmula escrita: `(SUMA(flujos) + CAPEX) / ABS(CAPEX)`, sin restar 1. La nuestra restaba, y el numerador ya incluía el flujo del año 0 con signo negativo, así que la inversión quedaba descontada dos veces. El síntoma estaba a la vista y no se había mirado: el ROI sin valor terminal daba −200 %, y perder más del 100 % de lo aportado no es posible. Corregido, **los tres umbrales se alcanzan en los dos cortes** y el apartado 8.9, que estaba escrito sobre la conclusión contraria, se rehízo entero.

Verificación

La entrega tiene su propio guion de verificación, y mira los archivos de la carpeta y no el libro de trabajo: `docs/negocio/_verificar-entrega-2.py`. Diez controles que comprueban, uno por uno, los cinco puntos de la devolución con las palabras con que la cátedra los pidió, los parámetros macro, los tres umbrales con su semáforo, el alcance y el racional de los desvíos. Se probó en rojo de tres maneras —sacando la hoja de capacidad, sacando el escenario pesimista y bajando la tasa de corte al 30 %— y las tres las detectó.

A eso se suman los 22 controles de `_verificar-alineacion.py`, los del propio libro —`docs/negocio/_verificar.py`, que ahora también comprueba que las 624 celdas de la proyección comercial sean enteras y que los 96 cierres mensuales cierren exactos— y la recalculación completa con Excel en cada paso. Dos guardas nuevas se probaron en rojo antes de darlas por buenas: el control de literales largos se corrió con el partidor desactivado y

nombró la celda, y el formato condicional de los umbrales se leyó del color que Excel efectivamente pinta y no del que debería pintar, con los dos estados presentes en la misma tabla.

E-13 · La segunda entrega de tecnología: el capítulo 10 contra el sistema

Fecha: 19 y 20/10/2026 · **Herramienta:** Claude Code

Tareas

Cerrar el sistema para la segunda entrega —la consigna es que lo que quede hasta la defensa sean ajustes y no funcionalidad nueva— y **cruzar el capítulo 10 contra tres fuentes que no son el documento**: el código, la base y el sistema andando. Más el cruce de los capítulos de negocio contra el 10, que es donde aparecen las promesas que el sistema no cumple. Siete deltas, D-98 a D-104.

Qué se aceptó

Las dos funciones que faltaban y el sistema prometía: el **alta de usuarios** —con contraseña temporal de un solo uso y cambio obligatorio al entrar— y el **respaldo heurístico de la IA**, para que la caída del proveedor externo no deje la clasificación y la guía vacías.

Y el **lector de código** (`docs/tecnico/_leer_codigo.py`), que extrae del fuente las clases, sus operaciones públicas y sus relaciones. Reemplaza el diagrama de clases dibujado a mano por cinco vistas generadas: un diagrama dibujado envejece sin avisar, uno leído no puede.

Qué decidió el autor

Entregar el sistema completo y no recortado, igual que en la primera entrega de tecnología y al revés de lo que se hizo con negocio. Allá la consigna enumeraba puntos y entregar de más habría sido entregar otra cosa; acá pide un piso y superarlo no cambia qué se entrega.

Corregir el documento en lugar de implementar lo prometido. El cruce con negocio encontró dos frases que prometían funciones inexistentes —las notificaciones por correo (6.6.1) y la importación desde planilla (5.2)—. Implementarlas a tres semanas de la entrega final era la decisión cara; declararlas fuera de alcance, con su lugar en el plan de producto, es la honesta.

Las credenciales de demostración no van en el LEEME de la entrega. La carpeta se publica en un repositorio público y el sistema está expuesto en internet: van en el mensaje a la cátedra.

Quedan, sí, en los archivos de prueba del fuente, porque sin ellas las pruebas de integración no corren.

Qué se corrigió de lo producido

Cuatro errores propios de la herramienta, los cuatro encontrados al verificar y ninguno por una prueba que fallara:

1. **El guión del capítulo 10 no era idempotente.** Buscaba su párrafo modelo por el texto «se presenta en **tres** vistas» y después reescribía ese mismo párrafo a «cinco», así que en la segunda corrida no se encontraba y reportaba como faltantes dos apartados —paquetes y componentes— que estaban aplicados. Un falso rojo es peor que un rojo: enseña a desconfiar del verificador. Se busca ahora por el prefijo que sobrevive a la reescritura.
2. **Los siete deltas no estaban escritos.** La cabecera del guión citaba «D-98 a D-104 de `CAMBIOS-DOCUMENTO.md`» y el archivo tenía 97: el trabajo del 19/10 se hizo y no se asentó, contra la regla de que el registro se actualiza en el momento. Se reconstruyeron contra los diffs reales, no de memoria.
3. **El armador escribió el LEEME de la primera entrega en la carpeta de la segunda**, con la tabla de contraseñas que justamente se había decidido sacar. Dos reemplazos de texto no habían coincidido y el guión siguió sin avisar. Se detectó leyendo el archivo generado, no la salida del comando.
4. **Un delta afirmaba que el sistema «notifica dentro de la aplicación»** y el documento dice que las notificaciones no están en esta versión. Se corrigió contra el texto del guión que aplica el cambio.

Verificación

401 pruebas de backend y 95 de frontend, todas en verde, y el documento de casos de prueba se regeneró de la corrida real —decía 362 y 91, de una corrida del 17/09—. Los 20 casos del capítulo 10 quedan cubiertos; ninguno se informa como faltante.

Las 152 pruebas que aparecieron en rojo no eran código roto, y vale la pena dejarlo escrito porque el diagnóstico fácil era el equivocado. Ciento ocho fallaban por una variable de entorno ausente —las de integración levantan la API en proceso y piden la contraseña de la base— y las otras 44 por una **migración pendiente**: el esquema agrega `debe_cambiar_contrasena` con un `ALTER` idempotente, pero la base local se había creado antes y nadie había vuelto a correr `aplicar.ps1`. El contenedor de la API, construido días antes, no consultaba esa columna y por eso el sistema andando no mostraba el problema.

De los 22 controles de alineación quedaban cuatro en rojo y se cerraron tres: el PDF del documento maestro estaba atrasado, tres diagramas regenerados no estaban dentro de ningún

documento —los `.docx` de los manuales se rehicieron— y la revisión OWASP declaraba 78 endpoints cuando hay 81. Los tres nuevos son los del alta de usuarios, y la revisión ahora dice qué patente exige cada uno. El cuarto, «ENTREGABLES contra sus fuentes», sigue en rojo a propósito: la carpeta de la entrega final se arma recién en noviembre.

Lo que se agregó el mismo día, después de armar la carpeta

Al revisar la entrega ya armada aparecieron tres cosas, y las tres se encontraron **cruzando dos fuentes que nadie había cruzado**, no leyendo mejor una sola:

1. **Las rutas del frontend contra el capítulo 10.5.** La pantalla de cambio de contraseña existía y no tenía sección. El control de pantallas estaba en verde porque salíaba toda ruta que no estuviera en un diccionario escrito a mano: sólo podía fallar por una pantalla ya registrada. Ahora una ruta desconocida falla.
2. **La captura de esa pantalla salió 404**, y ahí apareció lo más serio del día: **los contenedores eran del 17/09** y el sistema que estaba corriendo no tenía un mes de trabajo. Las pruebas no lo mostraban porque levantan la API en proceso con el código actual, no el contenedor.
3. **Los requisitos contra la evidencia**, que es la matriz de trazabilidad. Encontró que las tres pruebas de clasificación por IA corrían en verde sin que ningún caso de prueba las citara, y que RF-07b se apoyaba en el caso de la guía de reparación. Entró CP-21.

La matriz se construyó generada y no escrita, y con la comprobación en los dos sentidos: un requisito sin fila falla, y un caso de prueba que nadie reclama también. Lo segundo es lo que encontró el hueco. Se probó en rojo de tres maneras —un archivo inexistente, una clase que el archivo no nombra y un requisito sacado de la matriz— y las tres las detectó; la tercera, además, delató el caso de prueba que quedaba huérfano.

Lo que **no** se delegó acá: el mapeo de qué clase realiza cada requisito. Un programa puede comprobar que la clase existe, no que sea la que decide.

Qué no se delegó

Queda explícito, porque delimita el aporte propio del trabajo:

- **Las decisiones de arquitectura.** La herramienta aportó alternativas y consecuencias; la elección y su justificación son del autor y están firmadas en los ADR.
- **La definición del alcance.** Qué entra y qué no —en particular el alcance de la IA— lo definió el autor, descartando ampliaciones propuestas.
- **La validación.** Ninguna propuesta se dio por buena por parecer correcta: los scripts se ejecutan, las pruebas corren, y los defectos encontrados están asentados arriba.

- **El criterio sobre el resultado.** El caso del diseño de interfaz es el ejemplo: la salida de la herramienta se evaluó y se descartó.

Anexo B · Prompts críticos y validación técnica

Registro de las consultas asistidas por IA que afectaron decisiones de arquitectura o algoritmos del sistema, y del proceso por el cual cada propuesta se aceptó o se descartó.

Complementa `AI-USAGE-LOG.md`, que registra el uso general de herramientas. Acá va el detalle de las consultas que cambiaron el diseño.

Método de consulta

Las consultas de arquitectura se estructuran con el **modelo C4 de prompts**:

C1 · Contexto	Escenario, dominio y volumen esperado.
C2 · Constraints	Restricciones técnicas, infraestructura, tiempos, tecnologías predefinidas.
C3 · Criterio	Atributos de calidad prioritarios para *esta* decisión.
C4 · Output	Formato exacto de respuesta esperado.

Y se aplican tres patrones según el tipo de decisión:

- **Patrón A — Análisis de compromisos.** Nunca pedir una única solución: forzar la comparación de alternativas bajo atributos de calidad explícitos, con tabla matricial y recomendación justificada.
- **Patrón B — Abogado del diablo.** Antes de implementar, pedir la auditoría del diseño propio: puntos únicos de falla, cuellos de botella bajo carga incrementada y acoplamientos no deseados, con mitigación por patrones de resiliencia.
- **Patrón C — Registro de decisiones.** Formalizar la decisión resultante como ADR en formato Nygard.

> ***Sobre la forma de este registro.*** *Las consultas se sostuvieron de manera conversacional. Acá se documentan en forma estandarizada C4 para que sean comparables y reproducibles; el contenido —contexto, restricciones, criterios, alternativas evaluadas y validación— es el que efectivamente se manejó. De aquí en adelante las consultas de arquitectura se emiten directamente en este formato.*

PC-01 · Plataforma de desarrollo

Patrón A · Deriva en ADR 0002

C1 · Contexto. Sistema web de gestión y mantenimiento predictivo de activos informáticos para PyMEs. Volumen esperado: hasta 500 activos y 5000 incidencias por organización, decenas de usuarios concurrentes, sin picos significativos.

C2 · Constraints. El documento del TFI fija Java / Spring Boot / MySQL, pero esa definición no responde a ninguna restricción real. Existe un proyecto propio de referencia en C# con el modelo de capas que evalúa la cátedra. El sistema se defiende oralmente. No hay infraestructura preexistente.

C3 · Criterio. Correspondencia con el modelo de capas evaluado · capacidad del autor de sostener el código en la defensa · reutilización del andamiaje existente · costo de licencia.

C4 · Output. Tabla comparativa de tres alternativas bajo esos criterios, con recomendación justificada y consecuencias negativas de cada una.

Validación. Se leyó el código fuente del proyecto de referencia para confirmar que los patrones estaban efectivamente implementados y eran portables, en lugar de asumirlo desde su documentación. Se verificó que ningún requerimiento no funcional del documento (RNF-01 a RNF-07) dependiera de la plataforma.

Resultado. Aceptada la alternativa C# / ASP.NET Core. **Decisión del autor**, apartándose de lo que fija el documento.

PC-02 · Persistencia y acceso a datos

Patrón A · Deriva en ADR 0003 y ADR 0004

C1 · Contexto. El DER del documento ya separa negocio de servicio en dos diagramas. Hay que decidir si esa separación es física y con qué tecnología se accede.

C2 · Constraints. El Req. Arq. 002 exige bitácora inmutable y auditable. El Req. Arq. 003 exige respaldo y restauración. La cátedra evalúa la capa DAO y sus patrones.

C3 · Criterio. Aislamiento real entre dominio y seguridad · visibilidad de los patrones en el código · integridad referencial · costo de mantenimiento.

C4 · Output. Dos tablas comparativas —una por decisión— y el impacto de cada combinación sobre los requerimientos de arquitectura.

Validación. Se identificó que la combinación elegida **pierde integridad referencial entre bases** y que las operaciones que tocan las dos no son atómicas. Se aceptó explícitamente ese costo y se definió la mitigación: baja lógica de usuarios, validación en BLL, y bitácora asentada después del commit de negocio. Ambas consecuencias están escritas en el ADR 0003 en lugar de omitirse.

Resultado. Aceptadas dos bases separadas y ADO.NET con DAO explícito. Descartado EF Core por vaciar de contenido la capa que se evalúa.

PC-03 · Alcance de la integración con IA

Patrón A · Deriva en ADR 0006

C1 · Contexto. El documento define la asignación de incidencias por IA (RF-08, CU-007) con comportamiento de respaldo ante falla (CP-14). El usuario final que reporta es no técnico y describe los problemas en lenguaje coloquial.

C2 · Constraints. No hay crédito de API disponible. El sistema tiene que poder demostrarse sin conexión. CP-14 exige poder provocar la caída del servicio de forma reproducible. No se entrenan modelos propios (declarado en 6.4.1).

C3 · Criterio. Demostrabilidad sin dependencias externas · determinismo de los casos de prueba · que la IA no quede en el camino crítico como punto de falla · que el resultado sea siempre corregible por una persona.

C4 · Output. Alternativas de integración con su impacto sobre la reproducibilidad de CP-14, y propuesta de contrato de la interfaz.

Validación. Se verificó que la alternativa elegida hiciera CP-14 **determinístico** —condición para poder automatizarlo—. Se identificó un modo de falla que las propuestas iniciales no cubrían: el servicio puede responder correctamente pero nombrar un técnico inexistente o de otra organización. Se incorporó la validación de la respuesta contra el conjunto de técnicos reales como tercera protección.

Resultado. Aceptada la interfaz `IProveedorIA` con dos implementaciones. **El autor definió el alcance**: asignación y clasificación. Se descartaron dos ampliaciones propuestas —explicación del riesgo en lenguaje natural generada por IA, y asistente conversacional sobre el parque— por exceder lo que el documento sostiene.

PC-04 · Diseño del motor predictivo

Patrón A · Deriva en ADR 0007

C1 · Contexto. RF-11 define reglas configurables sobre el historial técnico que producen un score de riesgo por equipo. El documento nombra cinco condiciones con parámetros distintos entre sí.

C2 · Constraints. El administrador es no técnico. Cada tipo de regla necesita parámetros diferentes. El motor tiene que ser reevaluable y explicable: la pantalla de detalle muestra cuánto aportó cada regla.

C3 · Criterio. Que el motor sea determinístico y probable sin base ni red · que explique cada aporte al score · que agregar un tipo de regla no obligue a migrar el esquema · que no introduzca evaluación de entrada de usuario.

C4 · Output. Comparación de estrategias de modelado de los parámetros y del mecanismo de evaluación, con el impacto sobre la superficie de ataque.

Validación. Se descartó el lenguaje de expresiones editable por introducir ejecución de entrada de usuario sin beneficio para el perfil objetivo. Se verificó que el modelo elegido permitiera el preview en vivo de la pantalla de reglas —evaluar tiene que ser barato— y que `EvaluacionRiesgo` conservara la foto histórica para poder responder por qué un equipo estaba en rojo en el pasado.

Resultado. Aceptado: parámetros en JSON discriminados por `tipo`, resueltos con el patrón Strategy, con el conjunto de tipos restringido por `CHECK` en la base.

PC-05 · Auditoría del diseño antes de implementar

Patrón B — Abogado del diablo · Deriva en ADR 0008 y ADR 0009

C1 · Contexto. Arquitectura definida por los ADR 0002 a 0007, antes de escribir la capa DAO y la BLL.

C2 · Constraints. RNF-04 fija menos de 2 segundos con 500 activos y 5000 incidencias. RNF-05 fija 95 % de disponibilidad en horario laboral —es decir, alta disponibilidad **no** está en alcance— .

C3 · Criterio. Identificar puntos únicos de falla, cuellos de botella bajo un incremento del 300 % de la carga esperada, y acoplamientos temporales o estructurales no deseados.

C4 · Output. Hallazgos con su mitigación alineada a patrones de resiliencia.

Hallazgos

#	Hallazgo	Tipo	Mitigación
1	**El usuario espera a la IA.** CU-006 registra la incidencia y CU-007 se dispara sincrónicamente: con timeout de 15 s, el solicitante puede esperar 15 s para que le confirmen un ticket.	Acoplamiento temporal	**Circuit Breaker** sobre `IProveedorIA`: tras N fallos consecutivos el circuito abre y va directo al respaldo sin esperar el timeout. Evita que 20 incidencias seguidas paguen 15 s cada una. a ADR 0009
2	**El motor predictivo escala mal.** Su costo es del orden de equipos × reglas × historial. A 300 % de carga —tres veces el parque y tres veces el historial— es del orden de nueve veces el trabajo actual.	Cuello de botella	Evaluación incremental por equipo al registrar un evento; la evaluación del parque completo pasa a job programado fuera de horario. Índice por `(id_equipo, fecha)` ya presente en el esquema.
3	**N+1 al resolver el técnico de cada incidencia.** Al vivir en la otra base, una tabla de 50 incidencias podría disparar 50 consultas a la base de servicio.	Cuello de botella	Resolución **por lote**: una sola consulta por el conjunto de identificadores, y composición en la BLL. Es consecuencia directa del ADR 0003 y hay que sostenerla en cada listado.
4	**Instancia única de SQL Server con las dos bases.** Si cae, cae todo el sistema.	SPOF	Fuera del alcance del TFI por RNF-05 (95 %, no alta disponibilidad). Se mitiga con respaldos automáticos verificados y se **declara explícitamente** como límite conocido en lugar de simularlo resuelto.
5	**Resolución recursiva de permisos en cada petición.** El Composite Familia/Patente recorre el árbol para autorizar cada llamada.	Cuello de botella	**Caching** de permisos resueltos por sesión, invalidado al modificar roles o familias.

#	Hallazgo	Tipo	Mitigación
6	**Una evaluación larga puede consumir el pool de conexiones** y dejar sin atender a los usuarios.	Acoplamiento estructural	**Bulkhead**: pool de conexiones separado para los procesos en segundo plano. a ADR 0009
7	**El filtro por organización es un punto de falla de seguridad, no de disponibilidad.** Una consulta que lo omita filtra datos entre clientes.	Riesgo de seguridad	Filtro obligatorio en el DAO desde el contexto de sesión, pruebas de aislamiento por entidad, y revisión dirigida en SAST. a ADR 0005, ADR 0008

Descartado del análisis. Se evaluó y se descartó **Retry con backoff exponencial** sobre la llamada de IA en el camino crítico: reintentar multiplica la latencia que ya es el problema del hallazgo 1. El reintento queda sólo para la reevaluación en segundo plano, donde nadie está esperando.

Validación. Los hallazgos 2, 3 y 5 se van a verificar con las pruebas de carga definidas en el ADR 0008, no por inspección. El hallazgo 1 se verifica midiendo la latencia de registro de incidencia con el proveedor forzado a fallar, antes y después del circuit breaker.

Prompts de dirección del proyecto

Los PC-01 a PC-05 son consultas técnicas de diseño. Los que siguen son las **instrucciones que dirigieron el trabajo**: cada una fijó un rumbo, acotó un alcance o corrigió algo que la herramienta había dado por bueno. Se registran con el mismo formato porque cumplen la misma función de trazabilidad —dejar dicho quién decidió qué— y porque en varios casos son el punto donde el criterio del autor cambió el resultado.

PC-06 · Migración del stack y validación de la arquitectura base

Deriva en ADR 0002, ADR 0003 y ADR 0005

C1 · Contexto. Documento del TFI terminado, con el capítulo 10 especificando RF-01 a RF-14, CU-001 a CU-013 y CP-01 a CP-14. Cero líneas de código escritas. Se aportó como referencia el proyecto de tercer año de la carrera, con la separación de capas que evalúa la cátedra.

C2 · Constraints. Cuatro condiciones fijadas por el autor:

- El sistema tiene que ser **web**.

- El documento especifica Java, pero **se cambia a C#**, que es el lenguaje visto a lo largo de la carrera.
- La separación de carpetas del proyecto de referencia —servicio, BLL, domain, DAO— **se respeta**.
- Hay que **validar que estén** dos cosas: la configuración del cliente para la asignación de incidencias, y la separación en **dos bases de datos**, una de negocio y otra de seguridad y bitácora.

C3 · Criterio. El alcance documentado limita el análisis a reglas; la cátedra recomendó **integrar IA**, de modo que el alcance se amplía y el documento se modifica en consecuencia.

C4 · Output. Plan completo antes de escribir código, en modo de planificación. Y una condición sobre el método de trabajo: **antes de dar por terminada cualquier parte del sistema, correr las pruebas correspondientes.**

Resultado. Plan de ocho fases aprobado. Las cuatro condiciones se verificaron en la implementación: `ConfiguracionAsignacion` por organización, dos cadenas de conexión separadas, las cinco capas, y `IProveedorIA` para la IA. La condición sobre las pruebas es la que gobierna todo el registro de validación de este anexo.

PC-07 · Sincronización del documento con lo construido

Deriva en `docs/CAMBIOS-DOCUMENTO.md`

C1 · Contexto. El desarrollo empezó a apartarse del documento en varios frentes —lenguaje, alcance de la IA, modelo de datos— y las diferencias se estaban acumulando sin registro.

C2 · Constraints. Instrucción del autor: **adaptar el documento completo a lo que efectivamente se va a construir**, modificando alcance, integración de IA, lenguaje y base de datos. Las pantallas y los diagramas son **referencia, no especificación**: se ajustan a lo que el sistema termine siendo.

C3 · Criterio. Que cada desvío quede anotado **en el momento en que se produce**, con sección exacta, qué dice hoy, qué tiene que decir y por qué. No de memoria al final.

C4 · Output. Un registro incremental, no un informe final.

Resultado. `docs/CAMBIOS-DOCUMENTO.md` con quince entradas. Es el artefacto que permite responder en la defensa por qué el documento y el sistema dicen lo que dicen. Tres de esas entradas —D-13 a D-15— salieron de defectos que sólo aparecieron al usar el sistema, no al diseñarlo.

PC-08 · Entregables de transparencia sobre el uso de IA

Deriva en este anexo y ADR 0008

C1 · Contexto. Requisitos de la cátedra sobre transparencia en el uso de herramientas de IA, aportados por el autor.

C2 · Constraints. Tres entregables exigidos: anexo de trazabilidad —qué herramientas, para qué tareas, qué modificaciones manuales se aplicaron—, los prompts críticos con su validación técnica para aceptar o descartar, y un nivel de exigencia en **carga y seguridad superior al estándar tradicional**, precisamente por la velocidad que introduce el código asistido.

C3 · Criterio. El autor aportó además el **modelo C4 de prompts** —Contexto, Constraints, Criterio, Output— y tres patrones de consulta: **A** análisis de compromisos, **B** abogado del diablo, **C** registro de decisión en formato Nygard.

C4 · Output. Los tres entregables, con los patrones aplicados y señalables.

Resultado. Este anexo aplica el modelo C4 en todas sus entradas. El patrón B produjo la auditoría de PC-05, de la que salieron siete hallazgos y dos ADR nuevos. El patrón C es el formato de los nueve ADR. La exigencia elevada en carga y seguridad quedó en el ADR 0008, y es la razón por la que el aislamiento entre organizaciones tiene una prueba por operación y no una de muestra.

PC-09 · Aplicación de la devolución de cátedra

Deriva en D-10 de `docs/CAMBIOS-DOCUMENTO.md`

C1 · Contexto. Devolución de cátedra del 04/07/2026, aportada por el autor, que pide reforzar la justificación económica de los precios y vincularla con la estructura operativa.

C2 · Constraints. Aplicarla **al documento y también al sistema**.

C3 · Criterio. Que la justificación económica deje de ser una afirmación del texto y pase a ser algo que el sistema **calcula y muestra**.

C4 · Output. El cambio en las dos partes, y el aviso de si había que ajustar la especificación de diseño.

Resultado. Se incorporó `PlanComercial` al modelo de negocio. La pantalla de Organización muestra el abono, los equipos incluidos contra los administrados, los adicionales y el total mensual. Es el punto donde una afirmación del capítulo 6 se volvió una cuenta verificable en pantalla.

Corrección del autor sobre la herramienta. Al construir esa pantalla, la herramienta afirmó que con 52 equipos convenía el Plan Estándar. Verificado contra la base, el Inicial costaba ARS 101.500 y el Estándar ARS 102.900: el Estándar recién convenía desde 57 equipos. Con los precios revisados en D-14 la cuenta cambió de números pero no de conclusión: 244.400 contra 246.000, y el umbral pasa a 55. La corrección mejoró el sistema, porque la pantalla pasó a mostrar la cuenta real **incluso cuando el resultado es que no conviene cambiar al plan más caro**.

PC-10 · Construcción del sistema desde el diseño exportado

C1 · Contexto. El autor había descartado las pantallas construidas dentro de la sesión de desarrollo y redirigido el diseño a una herramienta específica. El resultado —veintiún artboards— se exportó al repositorio.

C2 · Constraints. Construir el sistema **a partir de ese diseño**, no de una interpretación propia. Y una instrucción sobre el método: **preguntar cualquier duda en lugar de asumir**.

C3 · Criterio. Que las pantallas implementadas se correspondan con los datos reales del sistema y no con datos de ejemplo.

C4 · Output. El sistema andando, verificado contra el diseño.

Resultado. Los tokens de diseño se tomaron literales del canvas exportado. Cada pantalla se recorrió en el navegador contra la base real. Ese recorrido encontró cuatro defectos que la lectura del código no había detectado, entre ellos que asignar un técnico no movía el estado de la incidencia, y que la consulta de técnicos devolvía cero resultados con un SQL que se leía correcto.

Corrección del autor sobre el diseño. El autor señaló que en la tabla «Equipos con mayor riesgo» del Dashboard, la columna rotulada **Responsable** traía nombres de técnicos. En esa tabla cada fila es un **equipo**, y un equipo no tiene técnico asignado: lo tienen sus incidencias. Se mantuvo el rótulo y se corrigió el dato, que es `Equipo.id_responsable`. Queda en D-15.

PC-11 · Presupuesto financiero y plan de negocio

Deriva en `docs/negocio/`

C1 · Contexto. El trabajo final se divide en dos mitades, negocio y sistema. La mitad de negocio necesita, además del documento, un modelo financiero.

C2 · Constraints. El autor aportó un presupuesto financiero real de otro trabajo como referencia de formato, y la lista de ítems exigidos: hipótesis, proyección de ventas por línea de producto, modelo de ingresos, modelo de egresos, costos fijos, costos variables y costos de RRHH. El plan de negocio, **en PDF sin excepción**.

C3 · Criterio. Modelo parametrizado, no una planilla de valores pegados: cada supuesto en una sola celda y el resto referenciándola por fórmula.

C4 · Output. Libro de diecisiete hojas con horizonte de cuatro años, mes a mes.

Resultado. El modelo dejó a la vista una inconsistencia del propio documento: la meta de 1.000 equipos al mes 12 no se alcanza con la mezcla de planes que proyecta el embudo. La desviación se dejó **visible en el bloque de verificación** de la hoja en lugar de ajustar los supuestos hasta que el número cerrara.

PC-12 · Revisión de los sueldos del presupuesto

Deriva en D-14 de `docs/CAMBIOS-DOCUMENTO.md`

Es el caso más claro de auditoría del autor sobre la salida de la herramienta, y el que más cambió el trabajo.

C1 · Contexto. Presupuesto financiero terminado y verificado, con los diecisiete tableros calculando sin errores de fórmula.

C2 · Constraints. Observación del autor: los precios del modelo parecían desactualizados y **los sueldos, bajos** — señaló que ARS 900.000 en varios roles le parecía poco.

C3 · Criterio. Contrastar las cifras contra el mercado real, no contra la coherencia interna del libro.

C4 · Output. Corrección de las cifras y del impacto en el resto del modelo.

Resultado. La observación era correcta y llevó a tres hallazgos encadenados que la verificación automática **no podía detectar**, porque el libro calculaba perfectamente:

1. Los ARS 900.000 equivalían a unos USD 600 mensuales: por debajo de un desarrollador junior. No eran una cifra desactualizada, sino —sin estar dicho en ninguna parte— el **retiro de subsistencia de los socios fundadores**. El modelo ahora separa el salario de mercado del porcentaje que los socios retiran, del 15 % en el año 1 al 100 % en el año 4 (los porcentajes se bajaron después, en D-33).
2. Buscando el origen apareció algo más grave: las filas macroeconómicas de la hoja de hipótesis estaban vacías, y eso dejaba los ingresos, los costos variables y los sueldos

planos los cuarenta y ocho meses, mientras los costos fijos escalaban porque tienen valores tipeados año por año. Tres de los cuatro bloques de un modelo a cuatro años en Argentina, nominalmente congelados.

3. Con sueldos a valor de mercado, **los precios del documento no sostenían el negocio en ningún año**: al cierre del año 4 el resultado operativo daba ARS −319 M y el equilibrio exigía 886 clientes contra los 352 que proyecta el embudo. El ingreso por cliente resultaba de unos USD 19 mensuales para administrar dos docenas de equipos, cuando las herramientas de RMM del mercado cobran entre USD 2 y USD 4 por equipo. Los precios revisados llevan el abono al rango de mercado, y con la recalibración posterior del calendario de incorporaciones —D-33— y la corrección de la línea del Segmento B —D-34— el año 4 cierra en ARS +220,7 M.

Lo que esto deja documentado. Un modelo puede estar internamente perfecto —cero errores de fórmula, todos los controles en verde— y sostener una premisa falsa. La verificación automática comprueba consistencia, no realismo. El realismo lo aportó la revisión del autor sobre una cifra que le resultó inverosímil.

PC-13 · Ejecución del sistema y cierre de la trazabilidad

C1 · Contexto. Sistema construido y probado, pero sin una forma simple de levantarlo para evaluarlo.

C2 · Constraints. Tres pedidos del autor: cómo ejecutar el sistema para probarlo, el registro de prompts **en un Word que se vaya actualizando**, y el estado real del documento. Además, que el anexo de trazabilidad y los manuales vayan en **archivos aparte** del cuerpo del documento.

C3 · Criterio. Que el anexo se pueda regenerar a medida que el registro crece, sin mantener dos versiones en paralelo.

C4 · Output. Script de arranque y generador del anexo.

Resultado. `arrancar.ps1` levanta base, API y frontend, y con `-IaCaida` fuerza la caída del proveedor para demostrar el CP-14 en vivo. El anexo se genera con `_generar-anexo.py` a partir de los dos markdown, que siguen siendo la fuente: se editan a mano y el Word se reconstruye entero.

PC-14 · Auditoría contra los requisitos de arquitectura de la cátedra

C1 · Contexto. Sistema funcionando, con las pruebas en verde y el documento sincronizado. Antes de darlo por cerrado, el autor pidió verificar que estuvieran cubiertos **todos** los puntos de

arquitectura del trabajo de tercer año: bitácora, seguridad, multiidioma, criptografía, patente-familia y patrones.

C2 · Constraints. El material de referencia —documento y script de base del proyecto de 3.º año— está en Drive y había que leerlo entero, no de memoria. Ninguna afirmación de cobertura sin verla contra el sistema.

C3 · Criterio. Que el resultado fuera una tabla de estado por requisito, con los huecos nombrados. Una auditoría que concluye «está todo bien» no auditó.

C4 · Output. Estado de los cinco Req. Arq. y de los siete CU.Arq.

Resultado. Cinco huecos reales, con distinto peso:

Requisito	Estado encontrado
Req. Arq. 001 · Multiidioma	**Ausente.** Todo el texto estaba escrito en los componentes.
Req. Arq. 002 · Bitácora	Parcial. La tabla y el disparador de inmutabilidad estaban; ver PC-15.
Req. Arq. 003 · Respaldos	Cubierto y verificado.
CU.Arq.006 · Criptografía	Parcial. Sólo hash de un sentido; faltaba cifrado reversible.
Patente-familia	Parcial. Faltaba el atributo `TipoAcceso` del modelo de referencia.

La lectura del script de referencia también sirvió para **confirmar una decisión anterior**: el proyecto de 3.º año no implementa dígito verificador, lo que respalda haberlo dejado fuera de alcance.

Se registró además una desviación deliberada: el modelo de referencia asocia patentes y familias directamente al usuario, sin tabla de roles. PredictIT inserta una capa de Rol, porque un sistema multi-organización necesita otorgar el mismo conjunto de permisos a muchas personas.

PC-15 · Verificación de la bitácora sobre datos reales

C1 · Contexto. La bitácora estaba implementada: tabla inmutable por disparador, servicio de escritura, pantalla de consulta y llamadas desde todos los business.

C2 · Constraints. Verificar sobre la base, no leyendo el código.

C3 · Criterio. Contar las filas por tipo de evento. Si la bitácora funciona, tiene que haber altas de incidencia, asignaciones y alertas, no sólo accesos.

C4 · Output. Conteo real por tipo de evento.

Resultado. El defecto más serio de toda la auditoría. La bitácora tenía 37 inicios de sesión, 6 accesos denegados, 4 respaldos… y **ningún evento de negocio**. Ocho de los nueve códigos que el sistema asentaba no existían en el catálogo `TipoEvento`, y el servicio, al no encontrar el código, avisa al logger y no escribe. Sin un solo error visible: ni excepción, ni respuesta distinta, ni prueba en rojo.

Era invisible leyendo el código, porque cada llamada estaba bien escrita. Sólo aparece contando filas.

Se completó el catálogo, se pasaron las llamadas a constantes —una constante mal escrita no compila, un literal sí— y se agregaron dos pruebas que sostienen la propiedad: una verifica por reflexión que cada constante declarada exista en la base, la otra rechaza literales sueltos en las llamadas a bitácora.

Al corregirlo apareció un segundo hueco del mismo tipo: la clasificación y la asignación automáticas **exitosas** tampoco dejaban rastro. Sólo se registraba el camino de respaldo, o sea el que falla. Ahora se registran las dos, distinguiendo si decidió la IA o una heurística: decir «IA» cuando resolvió una heurística falsearía justo la trazabilidad que la bitácora tiene que sostener.

PC-16 · El análisis predictivo no se aplicaba solo

C1 · Contexto. Durante la auditoría, el autor reportó que el sistema «no está aplicando análisis predictivo».

C2 · Constraints. El motor pasaba sus pruebas y la API devolvía datos correctos, así que el reporte parecía contradecir la evidencia.

C3 · Criterio. Antes de discutir el reporte, buscar quién invoca al motor.

C4 · Output. Lista de llamadores de `EvaluarParque` y `EvaluarEquipo`.

Resultado. El autor tenía razón y la evidencia no lo contradecía: el motor funcionaba, pero **nadie lo invocaba salvo el botón de la pantalla**. Ni el alta de una incidencia, ni el registro de un mantenimiento, ni ningún proceso programado.

Es la misma clase de defecto que PC-15 y merece señalarse como tal: **una pieza que falta no rompe nada**. Cada componente estaba bien, las pruebas ejercitaban el motor directamente, y lo que no existía no podía fallar. Los dos hallazgos más graves de esta auditoría fueron ausencias, no errores, y ninguno era detectable leyendo el código.

Se resolvió con dos disparadores —por hecho y por barrido programado— y quedó documentado en el ADR 0011.

PC-17 · Buscar lo declarado y no usado

C1 · Contexto. El sistema estaba dado por completo: los cinco Req. Arq. cubiertos, integración continua andando, y el documento sincronizado. El autor pidió verificar qué faltaba de código además del frontend.

C2 · Constraints. No responder de memoria ni con una lista de deseos: buscar en el código.

C3 · Criterio. La técnica que ya había encontrado dos defectos: comparar lo que el sistema **declara** contra lo que **usa**. Concretamente, recorrer las veintiséis patentes y los diecisiete códigos de evento y contar los usos reales de cada uno fuera de su declaración.

C4 · Output. La lista de los que no tenían ninguno.

Resultado. Cuatro patentes y dos códigos de evento sin un solo uso. Cada uno era un caso de uso que el modelo de permisos prometía y el sistema no tenía:

Declarado sin usar	Qué significaba
`INCIDENCIA_ATENDER`	La incidencia se registraba y se reasignaba, y nada más. Siete estados en el catálogo y sólo dos alcanzables
`EQUIPO_CAMBIAR_ESTADO`	Sólo existía dar de baja
`REPORTE_GENERAR`	RF-14 estaba en el documento y no en el sistema
`HISTORIAL_VER_ORGANIZACION`	La patente existía, la pantalla no
`LOGOUT`	La bitácora tenía todas las entradas y ninguna salida
`RESTORE`	El servicio sabía restaurar; ningún endpoint lo llamaba

El de `INCIDENCIA_ATENDER` era el grave: `diagnostico`, `solucion` y `fecha_resolucion` existían en la base, el detalle de la incidencia las devolvía, y ninguna consulta las escribía. Sin fecha de resolución no hay tiempo de reparación, y sin eso el SLA de cuatro horas que promete el documento no se puede medir.

Lo que este hallazgo dice del proceso. Es la **tercera vez** que la misma técnica encuentra algo, y las tres veces el hallazgo fue una **ausencia** y no un error: la bitácora sin eventos de negocio, el motor predictivo que no se disparaba solo, y ahora seis casos de uso que el modelo de permisos prometía. Ninguna prueba podía fallar, porque en los tres casos no había código roto.

De ahí sale la métrica que va primera en el plan de sprints, y de ahí sale que la Definición de Terminado exija verificar sobre el sistema corriendo.

PC-18 · El interbloqueo del botón «Resuelta»

C1 · Contexto. Implementado el ciclo de vida de la incidencia, con la máquina de estados en la capa de negocio y la pantalla consultando qué transiciones hay.

C2 · Constraints. El diseño era razonable y las pruebas pasaban: el negocio devolvía, junto con cada transición posible, el impedimento que la bloqueaba — «falta cargar el diagnóstico», «hace falta un técnico asignado»—, y la pantalla mostraba el botón deshabilitado con el motivo. Un botón que desaparece se lee como un error; uno deshabilitado que dice por qué se lee como una regla.

C3 · Criterio. Recorrer el flujo completo en la pantalla, no por la API.

C4 · Output. El estado real de los botones.

Resultado. El botón «Resuelta» quedaba deshabilitado **para siempre**. El negocio calculaba el impedimento sobre el diagnóstico **guardado**, y para guardar el diagnóstico hacía falta una transición, y la transición pedía el diagnóstico guardado. Un interbloqueo perfecto, y no había forma de verlo leyendo ninguno de los dos lados por separado: cada uno era correcto.

La corrección no fue un parche sino un reparto distinto de responsabilidades:

- **El negocio valida.** Rechaza resolver sin los dos textos, y esa es la única garantía: vale contra la API, contra Swagger y contra cualquier cliente futuro.
- **El negocio informa sólo los impedimentos que el formulario no puede conocer**: si hay técnico asignado y quién es.
- **El formulario guía**, evaluando lo escrito —que es lo que va a viajar en la misma llamada— y no lo guardado.

Quedó escrito en el propio código, en el comentario del método que calcula los impedimentos, porque es el tipo de decisión que alguien deshace por parecer una inconsistencia.

PC-19 · ¿El sistema es demasiado simple?

C1 · Contexto. Sistema completo y verificado, con el alcance del capítulo 10 cubierto y tres funciones de IA. Faltaban dos meses para la defensa.

C2 · Constraints. La duda era mía y no técnica: el trabajo podía parecer pobre en gestión al lado de un sistema de ventas con combos y facturación, como el de tercer año. La restricción que puse fue explícita: **no agregar pantallas por volumen**. Un módulo que existe para que el trabajo parezca más grande lo empeora, porque agrega superficie sin agregar sentido y hay que sostenerlo en la defensa.

C3 · Criterio. Buscar huecos que el propio sistema ya tuviera abiertos, y descartarlo si no había ninguno. Un módulo nuevo tenía que cerrar algo que el sistema promete y no cumple, no inventar una promesa nueva.

C4 · Output. Los huecos, con el lugar del sistema donde se ven.

Resultado. Aparecieron dos, y los dos eran reales:

1. **El motor predictivo recomendaba «programar el mantenimiento preventivo» y no había con qué programarlo.** La recomendación terminaba en un papel o en la memoria de alguien. `Mantenimiento` sólo registraba lo que ya había pasado.
2. **El plan de negocio cobra por abono más equipos administrados, y el sistema lo calculaba sin registrarlo.** La pantalla de Organización mostraba cuánto sale el mes; no había forma de saber qué se facturó, cuándo vence ni qué está impago. Era el puente que faltaba entre los capítulos 5 a 7 y el 10.

Los dos entraron. El criterio de selección quedó registrado porque es lo que distingue «agregué dos módulos» de «cerré dos huecos», y en la defensa la pregunta va a ser por qué están.

PC-20 · Con un plan cargado, ¿qué significa «vencido»?

C1 · Contexto. Construido el plan de mantenimiento, con sus fechas agendadas. La regla `MANTENIMIENTO_VENCIDO` del motor predictivo medía días desde el último preventivo contra un umbral escrito en la propia regla.

C2 · Constraints. Las dos definiciones conviven y se contradicen. Con un umbral de 180 días, un equipo que hace 250 días que no recibe un preventivo está «vencido» aunque la organización tenga el próximo agendado para la semana que viene. La regla estaría contradiciendo una decisión que alguien acaba de tomar sobre ese mismo equipo.

C3 · Criterio. Escribir primero las pruebas que describen el comportamiento deseado, verificar que fallen sobre el código actual, y recién entonces cambiar la regla. Y comprobarlo después contra el sistema corriendo, no sólo con dobles.

C4 · Output. Las pruebas en rojo, y después el cambio.

Resultado. Seis pruebas, dos en rojo. La regla pasa a medir contra el plan cuando el equipo tiene uno: «vencido» es el trabajo agendado cuya fecha ya pasó. El umbral no se va —es lo que sigue funcionando para los equipos sin plan, y es lo que satura el atraso, porque un mes de atraso pesa distinto sobre un plan mensual que sobre uno anual—, pero deja de decidir sobre los que sí tienen plan.

La comprobación sobre datos reales fue la que cerró la decisión:

Equipo	Antes	Ahora
`IMP-REC-002`	«hace 260 días del último preventivo»	«Preventivo estaba programado para el 08/08/2026 y lleva 34 día(s) de atraso»
`NB-COM-007`	vencido, por 200 días desde el último	no se cumple: «Preventivo programado para el 13/09/2026»

El segundo es el caso que el umbral suelto resolvía mal, y es el que justifica el módulo entero.

PC-21 · ¿Y si el prototipo tiene mal el contraste?

C1 · Contexto. El rediseño de la interfaz parte de `docs/design/prototipo-completo.html`, que fija paleta, tipografía y escala. La primera tarea era llevar la capa de tokens a esos valores.

C2 · Constraints. El RNF-07 compromete accesibilidad AA, y el proyecto la verifica con `docs/design/_contraste.py`, que mide cada token contra la superficie donde se dibuja. El prototipo es una maqueta suelta y nunca pasó por ese script.

C3 · Criterio. Aplicar los valores del prototipo y correr el script antes de dar el incremento por bueno, en vez de confiar en la maqueta.

C4 · Output. Los tokens alineados y el resultado del contraste.

Resultado. Tres pares por debajo del piso: `--linea-fuerte` sobre la superficie secundaria en tema oscuro (2,45:1), la misma sobre el lienzo en tema claro (2,82:1) y `--acento-hover` sobre blanco (3,59:1). Se corrigieron moviendo sólo la luminosidad y dejando el tono, porque la intención del prototipo es el color y lo que había que arreglar era cuánto se separa del fondo.

La decisión de fondo. El prototipo no manda sobre el RNF-07. Es la referencia de cómo se ve el sistema, no de qué promete cumplir.

PC-22 · ¿Qué le falta a la interfaz que no se ve mirándola?

C1 · Contexto. Terminado el rediseño de tablas y formularios. Todo lo visible del prototipo estaba aplicado.

C2 · Constraints. «Se ve bien» no es un criterio verificable, y revisar accesibilidad a ojo encuentra lo que uno ya sabe buscar. Se incorporó una base de reglas de UX con fuente y severidad —`ui-ux-pro-max`, instalada fuera del repositorio— para tener una lista que no salga de la propia memoria.

C3 · Criterio. Recorrer sus 48 reglas de severidad alta contra el sistema, y para cada una buscar la evidencia en el código en vez de responder de memoria.

C4 · Output. Qué está cubierto, qué no, y con qué archivo se comprueba.

Resultado. La mayoría ya estaba, y eso también es un resultado: esqueletos con `role="status"`, errores con `role="alert"`, estados vacíos con su acción, contraste verificado por script, `prefers-reduced-motion` respetado, contenido de IA rotulado, y el ingreso sin bloqueo de pegado ni de gestores de contraseñas.

Una no estaba: **el campo que el servidor rechaza no estaba enlazado con el mensaje que lo explica**. En pantalla se veía el borde rojo y el texto debajo, así que parecía resuelto; lo que faltaba —`aria-invalid` y `aria-describedby`— no se ve. Quien usa un lector de pantalla llegaba al campo, lo oía como cualquier otro, y el motivo quedaba suelto sin dueño.

Por qué entra acá. Es el caso que justifica haber traído la herramienta: lo que encontró no era un defecto visible, y ninguna de las revisiones anteriores —hechas mirando la pantalla— lo había levantado.

PC-23 · Una tarjeta vacía no se ve rota

C1 · Contexto. Movido el desglose del motor predictivo a una banda de ancho completo, la tarjeta aparecía con su título y sin contenido.

C2 · Constraints. La página en el navegador mostraba los datos; la captura que entra al documento, no. Dos observaciones que se contradicen.

C3 · Criterio. Medir en el navegador en vez de leer el CSS: pedir el alto calculado de la tarjeta y el de sus hijos.

C4 · Output. Los números, y de ahí la causa.

Resultado. La tarjeta medía **2 px** —sus dos bordes— con 400 px de contenido adentro. `.cuerpo` es un flex en columna: cuando el contenido no entra en la pantalla, reparte el faltante encogiendo a sus hijos, y una tarjeta lleva `overflow: hidden` para recortar sus esquinas, lo que le da mínimo automático cero. Las tablas de Activos e Incidencias venían recortadas por lo mismo desde antes, y nadie lo había visto: se leía como «la tabla termina ahí».

Lo que se llevó de esto. Que el defecto no se vea roto es lo que lo hace peligroso: una tarjeta vacía con su título entra al documento como si fuera el sistema. La guarda quedó donde único puede vivir —el script de capturas, que es el que tiene un navegador de verdad—: después de

cada pantalla mide si alguna tarjeta es más baja que su contenido y termina en rojo. Comprobada provocando el colapso a propósito.

PC-24 · ¿La plantilla alcanza para atender la cartera del año 4?

C1 · Contexto. La devolución de la Entrega 1 lo pidió así: «deberá justificarse cuantitativamente la capacidad operativa de la estructura prevista al cierre del horizonte. La relación LTV/CAC demuestra eficiencia comercial, pero no capacidad de soporte». La cartera al cierre es de 353 clientes y 9.370 equipos con seis personas, de las cuales dos atienden.

C2 · Constraints. No vale responder con una afirmación cualitativa, y tampoco vale elegir los parámetros de carga de manera que la respuesta sea que sí. Los supuestos tienen que ser defendibles uno por uno y vivir en la hoja de Hipótesis como cualquier otro.

C3 · Criterio. Construir la cadena completa —clientes a atender, incidencias, deflección por autoservicio, minutos por incidencia, horas de onboarding, horas productivas por persona— y **mirar el resultado antes de decidir si el calendario de RRHH estaba bien**. Si la medición contradice al plan, se corrige el plan.

C4 · Output. La serie mensual de personas necesarias contra personas en la plantilla, y la holgura.

Resultado. La plantilla no alcanzaba. La carga supera a una persona en el **mes 28** y el segundo colaborador de soporte estaba previsto para el **42**: catorce meses de sobrecarga, justo cuando la cartera pasa de 157 a 278 clientes. El alta se adelantó al mes que la medición indica y el costo quedó declarado: 17,9 millones de valor actual neto y 34,6 de capital, que es además lo que devolvió el escenario pesimista a terreno negativo.

Lo que se llevó de esto. El orden importa más que la respuesta. La misma hoja, construida después de fijar el calendario, habría terminado con los parámetros ajustados hasta que la plantilla alcanzara —y nadie lo habría notado, porque ninguno de los seis supuestos es verificable desde afuera—. Lo que la vuelve defendible es que el número que cambió fue el del calendario. Quedan cinco meses de déficit al cierre del horizonte y están escritos en 7.6 en lugar de resueltos con una tercera contratación que el modelo no sostiene.

PC-25 · ¿A qué corte se miden los umbrales de aprobación?

C1 · Contexto. La cátedra fijó tres condiciones simultáneas para el cierre de la evaluación financiera: valor actual neto positivo, tasa interna de retorno por encima del 40 % y retorno acumulado sobre la inversión por encima del 120 %. El enunciado dice dos cosas sobre cuándo

medirlas: los targets son «al cierre del Año 3» y, unas líneas antes, «el formato está hecho para un marco temporal de 3 años, si sus negocios tienen más tiempo ajústenlo», con la condición evaluada «al finalizar el período evaluado».

C2 · Constraints. Nuestro horizonte es de cuatro años. Los dos cortes no son equivalentes y la diferencia no es menor: el año 4 es el ejercicio en que el resultado operativo se da vuelta.

C3 · Criterio. Calcular los tres indicadores en los dos cortes —con el valor terminal recalculado sobre los ingresos del año que corresponda, no movido de columna— y recién después decidir qué dice el documento.

C4 · Output. Los seis números, con su estado contra cada umbral.

Resultado. Los tres umbrales se alcanzan en los dos cortes: 235,1 millones, 126 % y 653 % al cierre del año 4, y 86,1 millones, 111 % y 192 % al del año 3. El documento declara los dos y explica la distancia entre ellos, que es el año en que el proyecto se da vuelta.

La primera versión de esta respuesta decía otra cosa, y corresponde asentarlo: daba 92 % de retorno al año 3 y concluía que el proyecto no alcanzaba el umbral. El error era propio —la fórmula del ROI restaba 1 de más, descontando la inversión dos veces— y lo destapó la plantilla que la cátedra publicó después, que trae la fórmula escrita. El síntoma estaba a la vista desde el principio: el ROI sin valor terminal daba −200 %, y perder más del 100 % de lo aportado no es posible. No se lo había mirado porque el número que importaba parecía razonable.

Lo que se llevó de esto. Con un enunciado ambiguo, elegir la lectura conveniente y no mencionar la otra es indistinguible de no haberla visto, y en una defensa esa ambigüedad la pregunta el jurado. La hoja además mide los tres indicadores **sin valor terminal**, donde ninguno alcanza su umbral en ninguno de los dos cortes, y lo hace por fórmula: si algún día dejara de ser cierto, el libro lo diría solo en lugar de que el texto quede mintiendo. El verde de la columna de estado lo pinta Excel con formato condicional por el mismo motivo —un color escrito en el archivo sobrevive al cambio de hipótesis que lo vuelve falso—, y se verificó leyendo el color que Excel efectivamente aplica, con los dos estados presentes en la misma tabla.

Consultas cuya propuesta se descartó

Se registran porque la decisión de no hacer algo también es una decisión.

Propuesta	Motivo del descarte
Explicación del riesgo en lenguaje natural generada por IA	Excede el alcance de IA que sostiene el documento. El motor ya explica su razonamiento con los datos concretos de cada regla, sin necesidad de un modelo de lenguaje.

Propuesta	Motivo del descarte
Asistente conversacional sobre el parque informático	Misma razón, y agrega superficie de ataque —traducción de lenguaje natural a consultas— sin resolver un problema declarado.
Entity Framework Core	Vacía de contenido la capa DAO y el Unit of Work, que son objeto de evaluación.
Lenguaje de expresiones editable para las reglas	Introduce evaluación de entrada de usuario. El perfil objetivo no lo usaría.
Dígito verificador horizontal y vertical	No está entre los Req. Arq. 001 a 005 del documento ni lo implementaba el proyecto de referencia.
Replicar la estructura documental completa del proyecto de 3.º año	El autor precisó que no se exige esa estructura; los manuales sí.
Construir las 16 pantallas dentro de la sesión de desarrollo	El autor evaluó el resultado como insuficiente y redirigió a una herramienta de diseño específica.
Retry con backoff sobre la llamada de IA en el camino crítico	Multiplica la latencia que ya es el problema a resolver.
Tabla propia de errores del sistema, aparte de la bitácora	Un segundo registro de lo mismo se desincroniza del primero. La bitácora ya guarda la traza y ya es inmutable.
Cifrar también las contraseñas, para tener un solo mecanismo criptográfico	Sería una regresión de seguridad: volvería recuperable algo que no tiene por qué serlo. La uniformidad no es un valor cuando los problemas son distintos.
Un contexto de «superusuario» para el barrido predictivo programado	Un proceso automático con todos los permisos es un proceso que puede hacer cualquier cosa el día que tenga un error. Lleva sólo la patente que necesita.
CSV como formato de los reportes exportables	Lo descartó el autor. Un CSV no lleva encabezado, no lleva los filtros impresos y no se ve como un entregable, y el reporte tiene que poder explicarse solo cuando circula fuera del sistema.
Generar el PDF en el navegador	El reporte lleva impresos la fecha, el autor y los filtros, y eso sólo es creíble si lo escribe el sistema. Además el servidor no se enteraría de la emisión, y la bitácora tiene que saber que un reporte salió.
Un catálogo de transiciones configurable en base	Sería más flexible y permitiría dejar el sistema en un estado sin salida por configuración. Las transiciones de una incidencia no son algo que cada organización deba poder cambiar.
Subir el múltiplo del valor terminal hasta que el VAN cierre en positivo	Es elegir el número que hace que la respuesta salga bien. Se toma el piso del rango y el resultado negativo se declara, con la sensibilidad al lado.